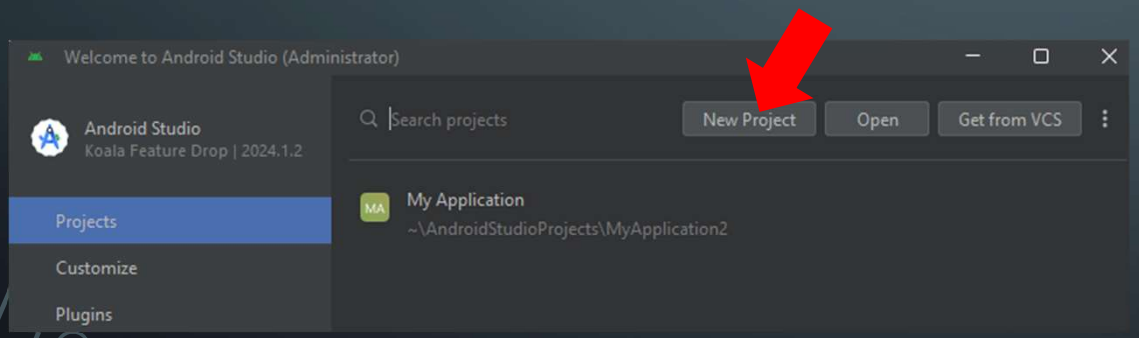
DESENVOLVIMENTO FULL STACK

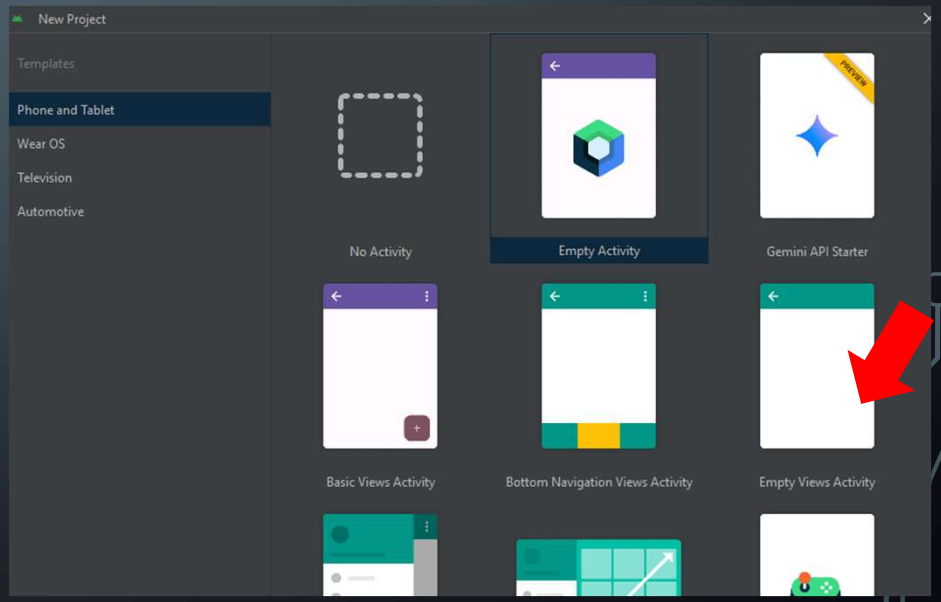
ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

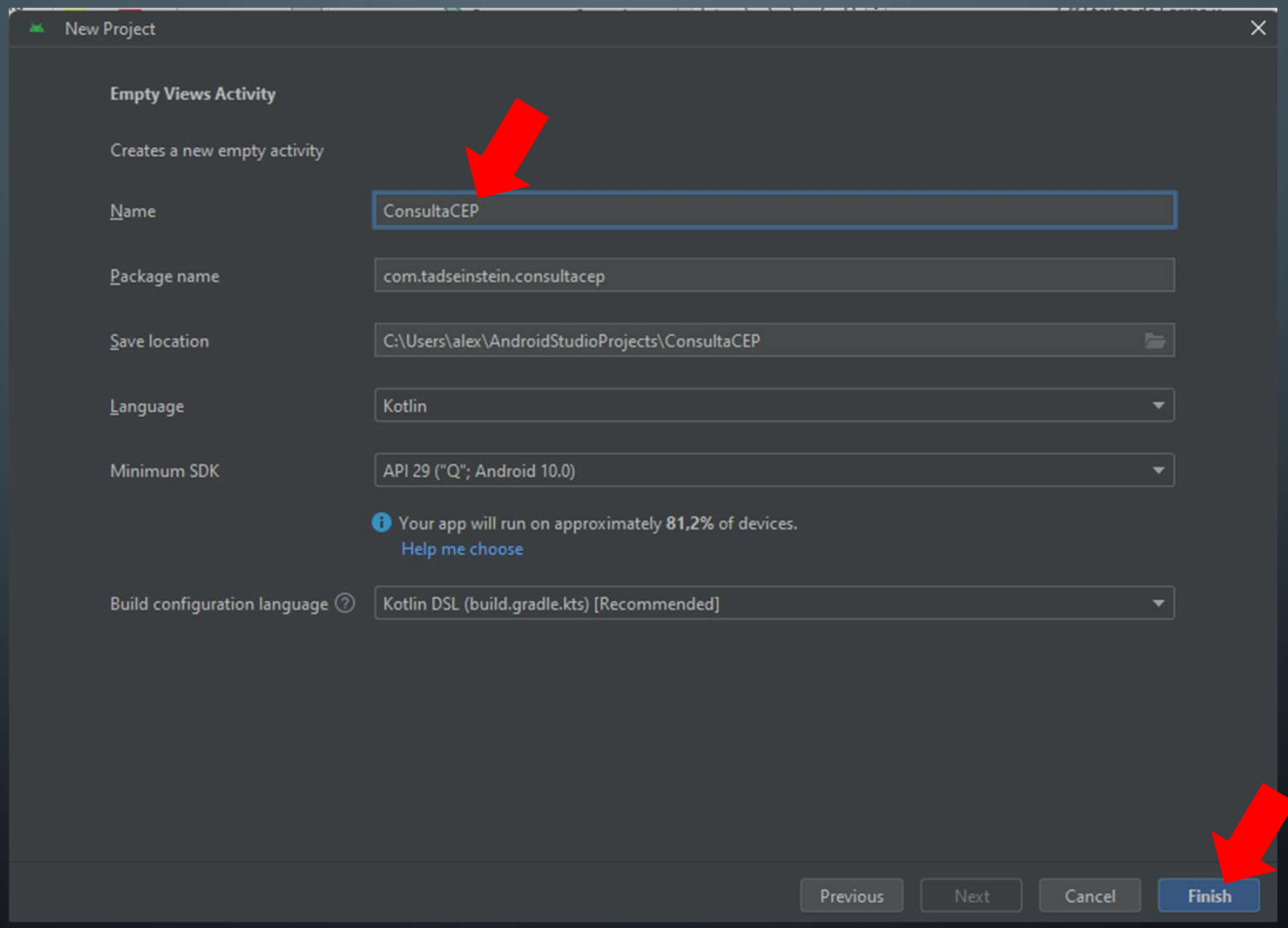
ACESSANDO API DE CEP NO ANDROID STUDIO

- Acessando a API da ViaCEP pelo Android Studio, vamos buscar o endereço de um CEP informado.
- Abra o Android Studio e crie um novo projeto





ACESSANDO API DE CEP NO ANDROID STUDIO



ACESSANDO API DE CEP NO ANDROID STUDIO

- Vamos começar desenhando a tela, no arquivo activity_main.xml (esta na pasta res\layout) apague o TextView que ele inclui por padrão, troque o layout para LinearLayout e coloque a orientação vertical:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

</LinearLayout>
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Montando a tela:

```
11    <EditText
12        android:id="@+id/edit_cep"
13        android:inputType="number"
14        android:layout_marginTop="30dp"
15        android:layout_width="130dp"
16        android:layout_height="wrap_content"
17        android:layout_gravity="center_horizontal"
18        android:textSize="20sp"
19        android:hint="Digite o CEP"/>
20
21    <TextView
22        android:layout_width="wrap_content"
23        android:layout_height="wrap_content"
24        android:layout_gravity="center_horizontal"
25        android:textSize="20sp"
26        android:textStyle="bold"
27        android:text="CEP"/>
28
```

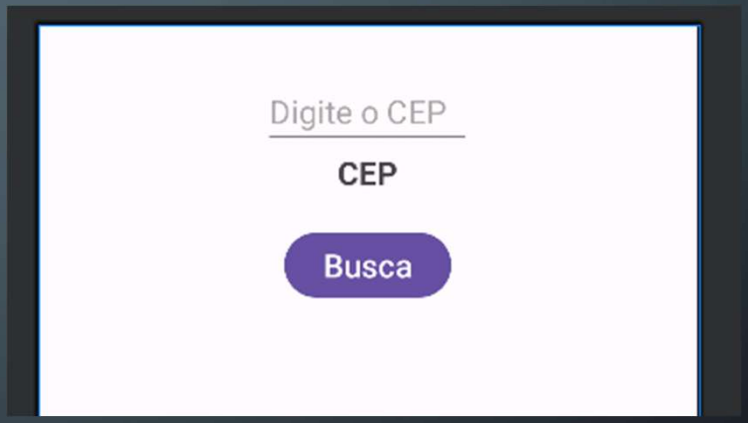


- Lembre-se que a melhor forma de programar para android é colocando todos os textos que serão escritos na tela no arquivo strings.xml, não estamos fazendo isso apenas para agilizar o desenvolvimento.

ACESSANDO API DE CEP NO ANDROID STUDIO

- Vamos incluir um botão para buscar o CEP:

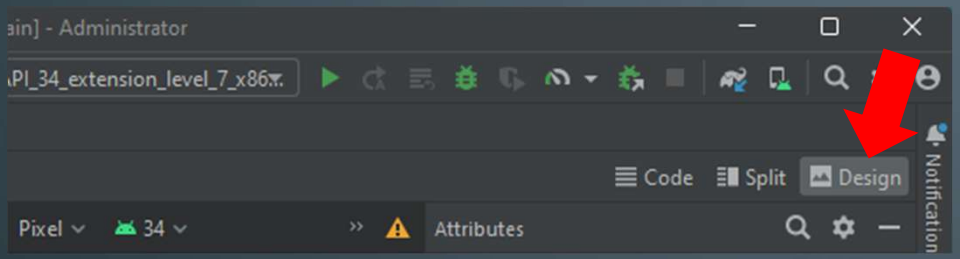
```
29    <Button
30        android:id="@+id/button_procuraCEP"
31        android:layout_width="wrap_content"
32        android:layout_height="wrap_content"
33        android:layout_gravity="center_horizontal"
34        android:text="Busca"
35        android:layout_marginTop="20dp"
36        android:textSize="20sp"/>
```

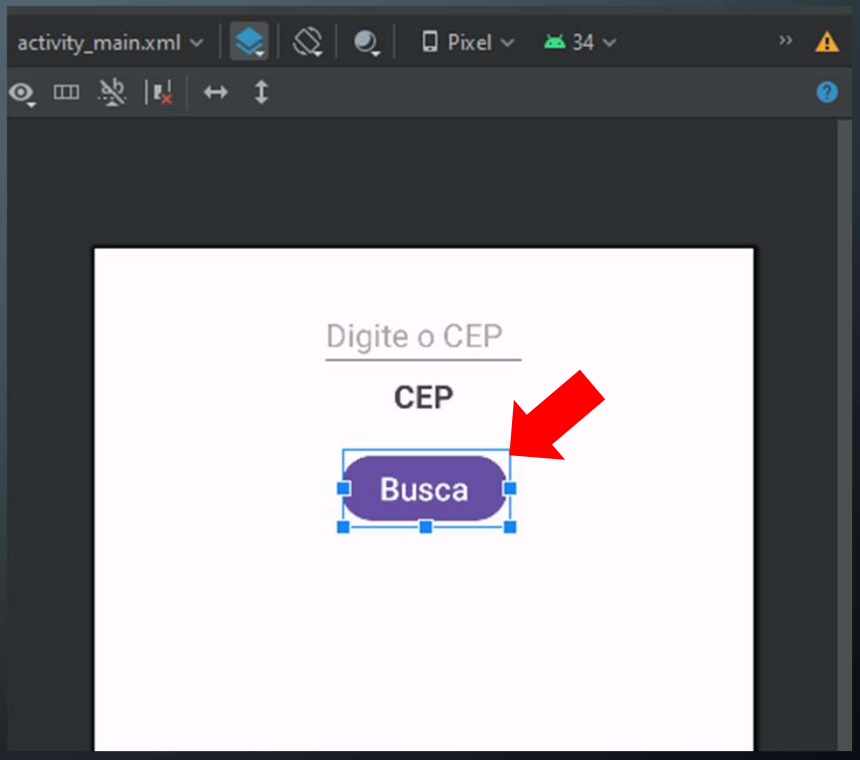


- Vamos incluir um ícone de procura nesse botão

ACESSANDO API DE CEP NO ANDROID STUDIO

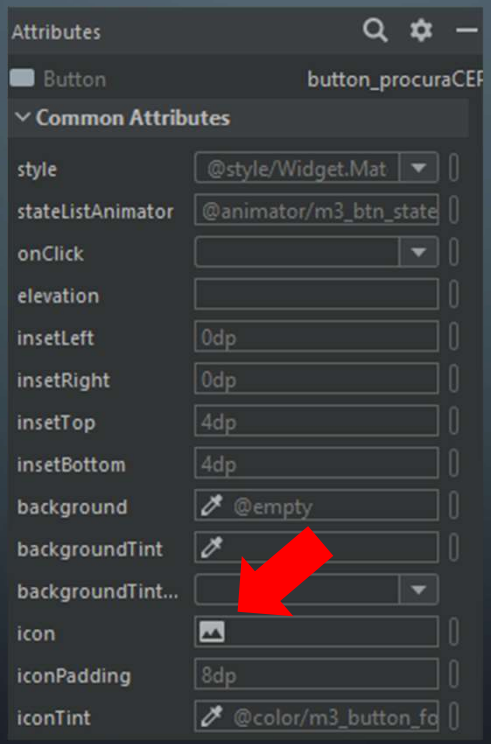
- Clique em Design e clique sobre o botão que esta na tela:





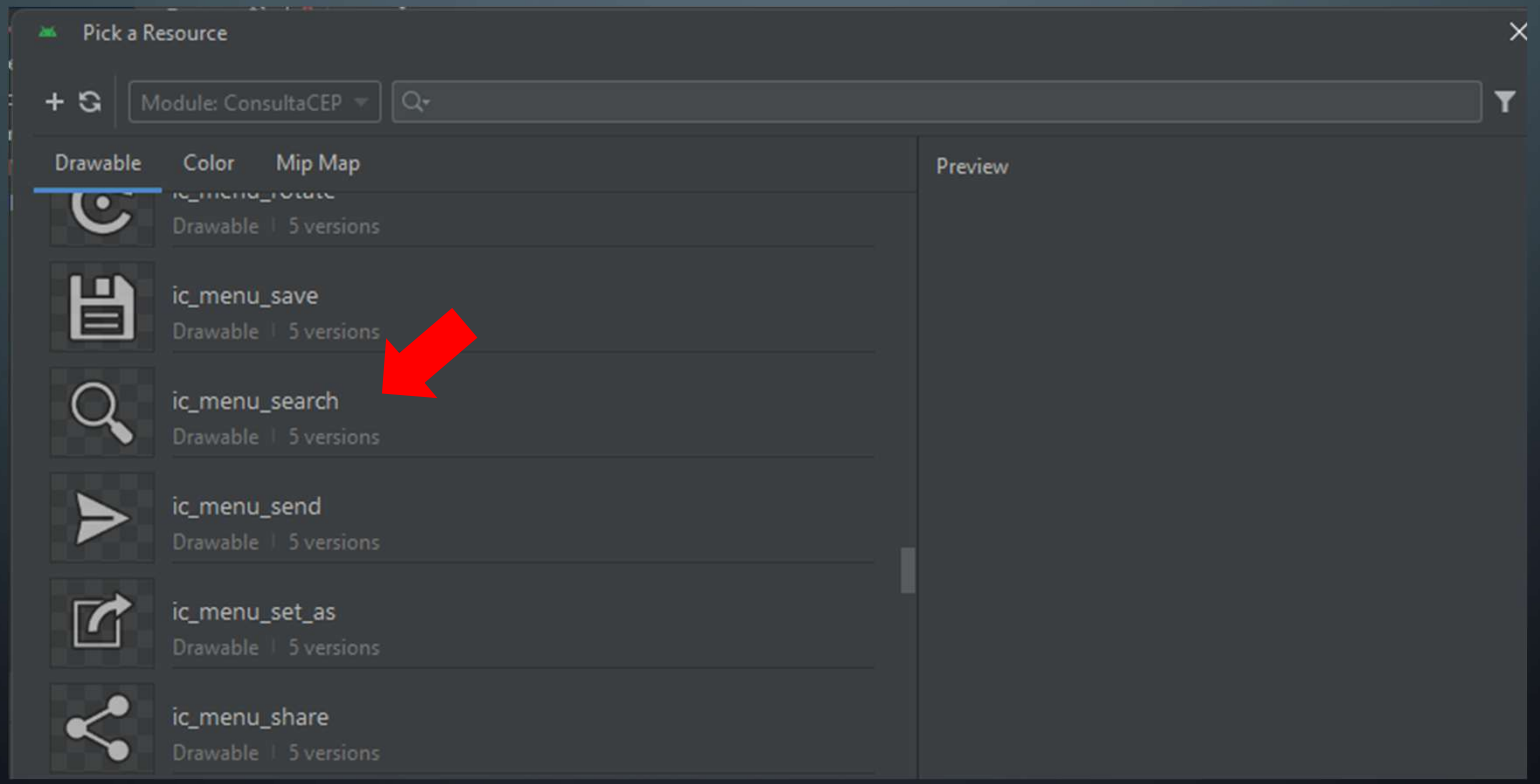
ACESSANDO API DE CEP NO ANDROID STUDIO

- Na janela de atributos que vai aparecer no lado direito, desça até a opção Icon e clique sobre o desenho ao lado:



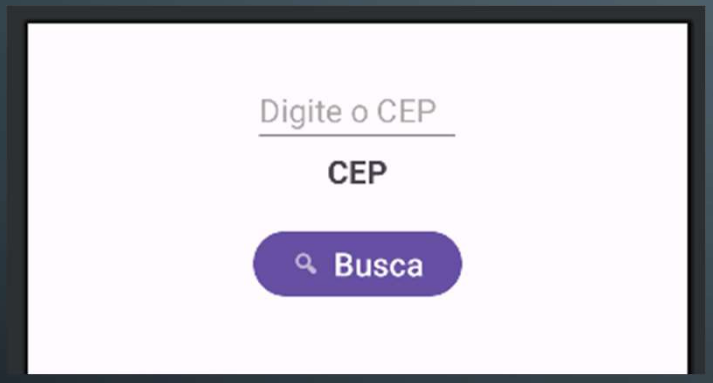
ACESSANDO API DE CEP NO ANDROID STUDIO

- Localize e selecione o ícone ic_menu_Search:



ACESSANDO API DE CEP NO ANDROID STUDIO

- Note que ele incluiu o ícone antes do texto no botão, e colocou uma entrada no .xml de desenho da tela:




```
29      <Button
30          android:id="@+id/button_procuraCEP"
31          android:layout_width="wrap_content"
32          android:layout_height="wrap_content"
33          android:layout_gravity="center_horizontal"
34          android:layout_marginTop="20dp"
35          android:text="Busca"
36          android:textSize="20sp"
37          app:icon="@android:drawable/ic_menu_search" />
38
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Agora vamos incluir os campos que serão preenchidos com os dados retornados pela API:

```
39      <EditText
40          android:id="@+id/edit_rua"
41          android:inputType="number"
42          android:layout_marginTop="60dp"
43          android:layout_width="match_parent"
44          android:layout_height="wrap_content"
45          android:layout_gravity="center_horizontal"
46          android:textSize="20sp"
47          android:hint="Digite a rua"/>
48
49      <TextView
50          android:layout_width="wrap_content"
51          android:layout_height="wrap_content"
52          android:layout_gravity="center_horizontal"
53          android:textSize="20sp"
54          android:textStyle="bold"
55          android:text="Logradouro"/>
56
```



ACESSANDO API DE CEP NO ANDROID STUDIO

- O número e complemento iremos colocar um na frente do outro, para isso vamos utilizar outro linearLayout, mas como queremos colocar um texto abaixo como fizemos nos outros campos será necessário utilizar três linearLayout, um na horizontal e dois na vertical:

ACESSANDO API DE CEP NO ANDROID STUDIO

- A estrutura dos linearLayout ficará assim:

```
57      <LinearLayout
58          android:layout_width="match_parent"
59          android:layout_height="wrap_content"
60          android:orientation="horizontal">
61
62          <LinearLayout
63              android:layout_width="0dp"
64              android:layout_weight="1"
65              android:layout_height="wrap_content"
66              android:orientation="vertical">
67
68
69          </LinearLayout>
70
71          <LinearLayout
72              android:layout_width="0dp"
73              android:layout_weight="1"
74              android:layout_height="wrap_content"
75              android:orientation="vertical">
76
77
78          </LinearLayout>
79
80
81      </LinearLayout>
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Agora vamos incluir os componentes dentro dos linearLayout que criamos:

```
66          android:orientation="vertical">
67
68          <EditText
69              android:id="@+id/edit_numero"
70              android:inputType="number"
71              android:layout_width="100dp"
72              android:layout_height="wrap_content"
73              android:layout_gravity="center_horizontal"
74              android:textSize="15sp"
75              android:hint="numero"/>
76
77          <TextView
78              android:layout_width="wrap_content"
79              android:layout_height="wrap_content"
80              android:layout_gravity="center_horizontal"
81              android:textSize="20sp"
82              android:textStyle="bold"
83              android:text="Número"/>
84
85      </LinearLayout>
86
```

```
91          android:orientation="vertical">
92
93          <EditText
94              android:id="@+id/edit_complemento"
95              android:inputType="number"
96              android:layout_width="180dp"
97              android:layout_height="wrap_content"
98              android:layout_gravity="center_horizontal"
99              android:textSize="15sp"
100             android:hint="digite o complemento"/>
101
102         <TextView
103             android:layout_width="wrap_content"
104             android:layout_height="wrap_content"
105             android:layout_gravity="center_horizontal"
106             android:textSize="20sp"
107             android:textStyle="bold"
108             android:text="Complemento"/>
109
110     </LinearLayout>
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Resultado:



- Agora insira os campos que estão faltando:

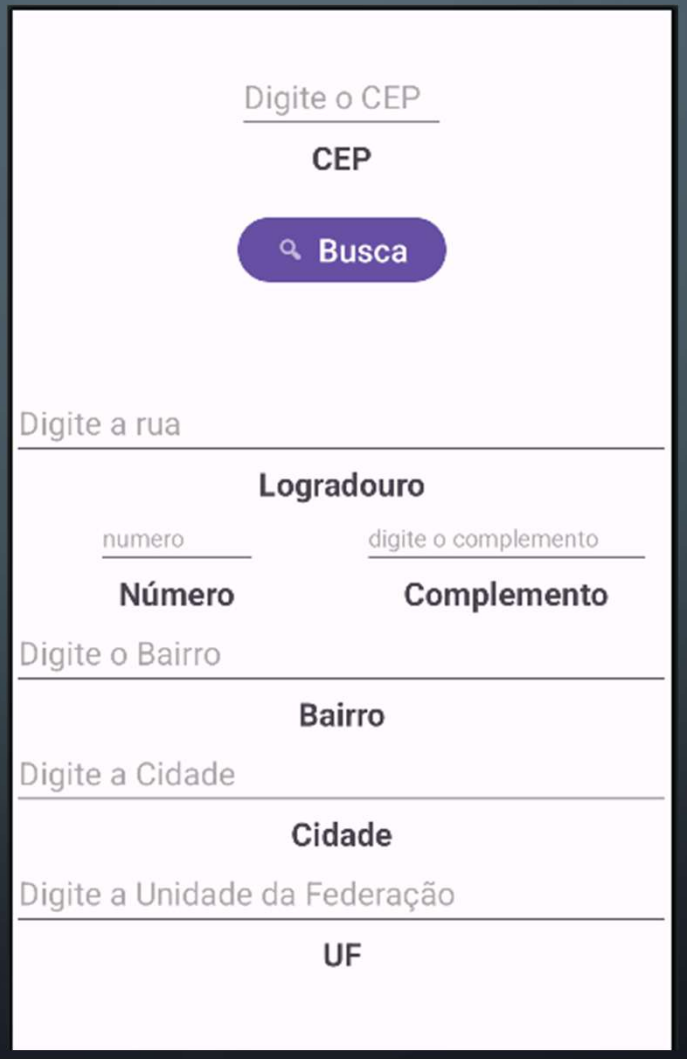
ACESSANDO API DE CEP NO ANDROID STUDIO

```
115     <EditText
116         android:id="@+id/edit_bairro"
117         android:inputType="number"
118         android:layout_width="match_parent"
119         android:layout_height="wrap_content"
120         android:layout_gravity="center_horizontal"
121         android:textSize="20sp"
122         android:hint="Digite o Bairro"/>
123
124     <TextView
125         android:layout_width="wrap_content"
126         android:layout_height="wrap_content"
127         android:layout_gravity="center_horizontal"
128         android:textSize="20sp"
129         android:textStyle="bold"
130         android:text="Bairro"/>
131
132     <EditText
133         android:id="@+id/edit_cidade"
134         android:inputType="number"
135         android:layout_width="match_parent"
136         android:layout_height="wrap_content"
137         android:layout_gravity="center_horizontal"
138         android:textSize="20sp"
139         android:hint="Digite a Cidade"/>
```

```
141     <TextView
142         android:layout_width="wrap_content"
143         android:layout_height="wrap_content"
144         android:layout_gravity="center_horizontal"
145         android:textSize="20sp"
146         android:textStyle="bold"
147         android:text="Cidade"/>
148
149     <EditText
150         android:id="@+id/edit_uf"
151         android:inputType="number"
152         android:layout_width="match_parent"
153         android:layout_height="wrap_content"
154         android:layout_gravity="center_horizontal"
155         android:textSize="20sp"
156         android:hint="Digite a Unidade da Federação"/>
157
158     <TextView
159         android:layout_width="wrap_content"
160         android:layout_height="wrap_content"
161         android:layout_gravity="center_horizontal"
162         android:textSize="20sp"
163         android:textStyle="bold"
164         android:text="UF"/>
```

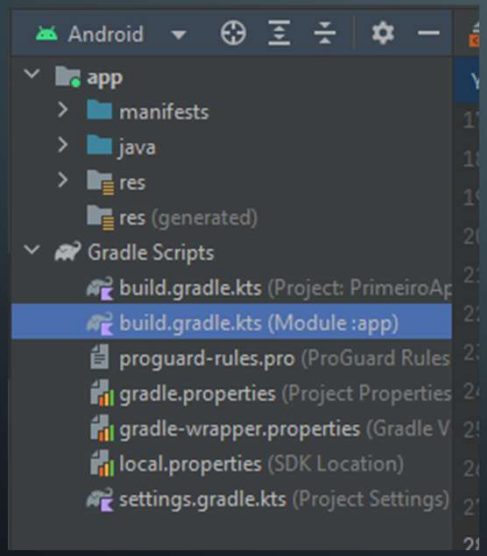
ACESSANDO API DE CEP NO ANDROID STUDIO

- Tela completa:



ACESSANDO API DE CEP NO ANDROID STUDIO

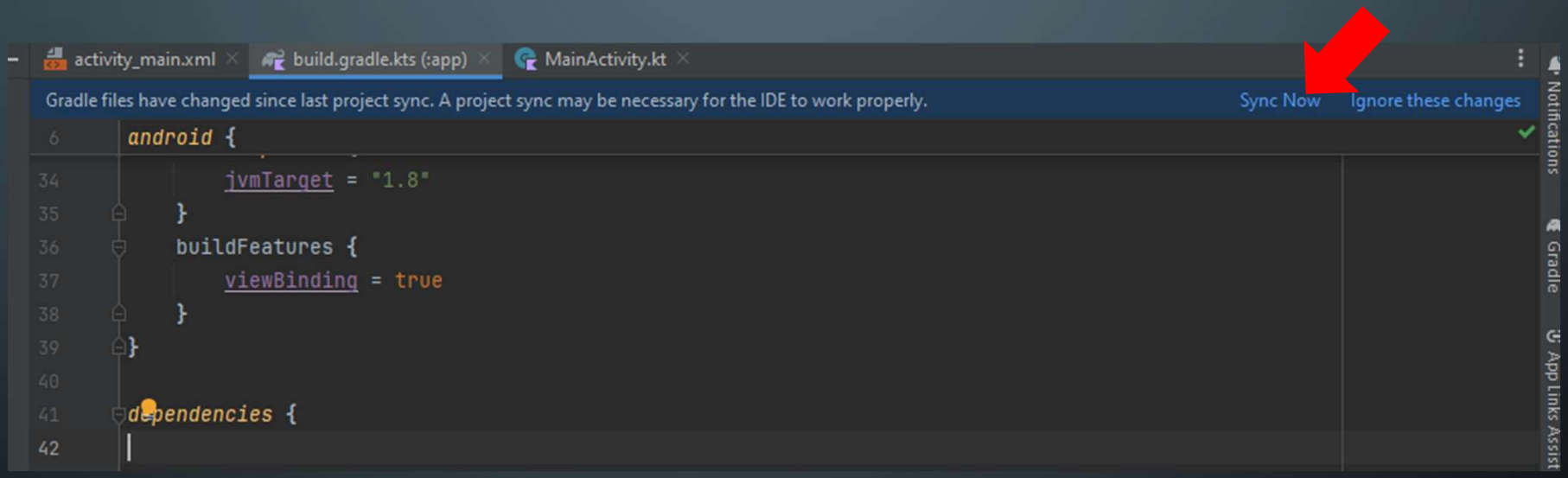
- Visual terminado, precisamos fazer o ViewBinding. Edite o arquivo buidgradle.kts(Module:app) da pasta Grade Scripts e inclua o código abaixo:




```
28      }
29      compileOptions {
30          sourceCompatibility = JavaVersion.VERSION_1_8
31          targetCompatibility = JavaVersion.VERSION_1_8
32      }
33      kotlinOptions {
34          jvmTarget = "1.8"
35      }
36      buildFeatures {
37          viewBinding = true
38      }
39  }
40
41  dependencies {
```

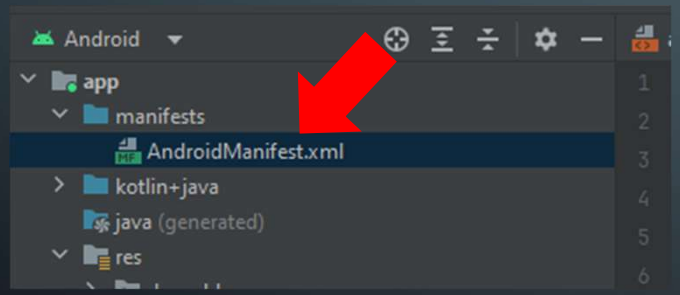
ACESSANDO API DE CEP NO ANDROID STUDIO

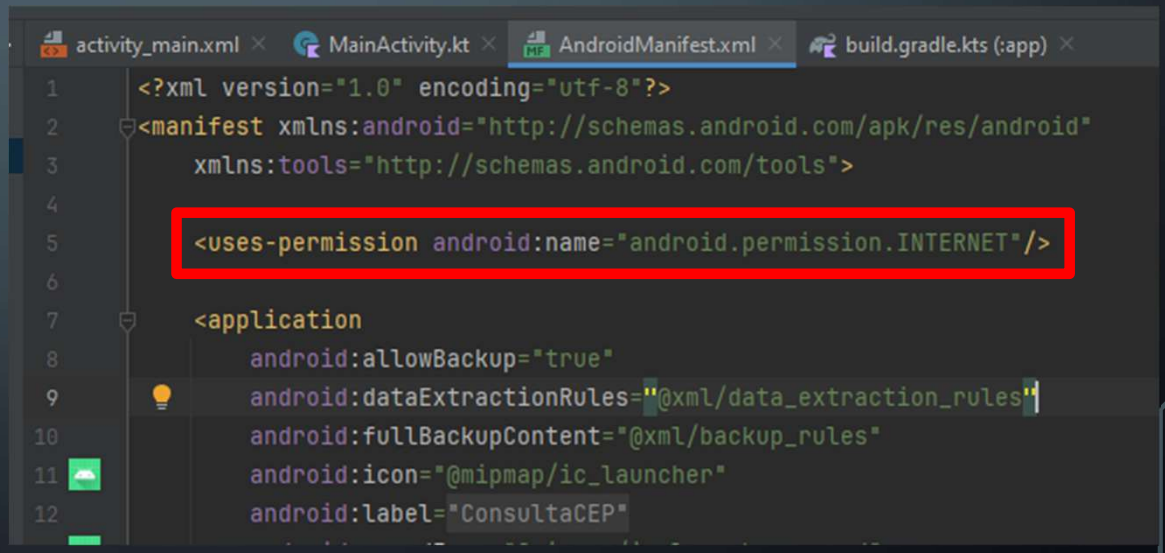
- Após a alteração, clique em Sync Now para validar as alterações.



ACESSANDO API DE CEP NO ANDROID STUDIO

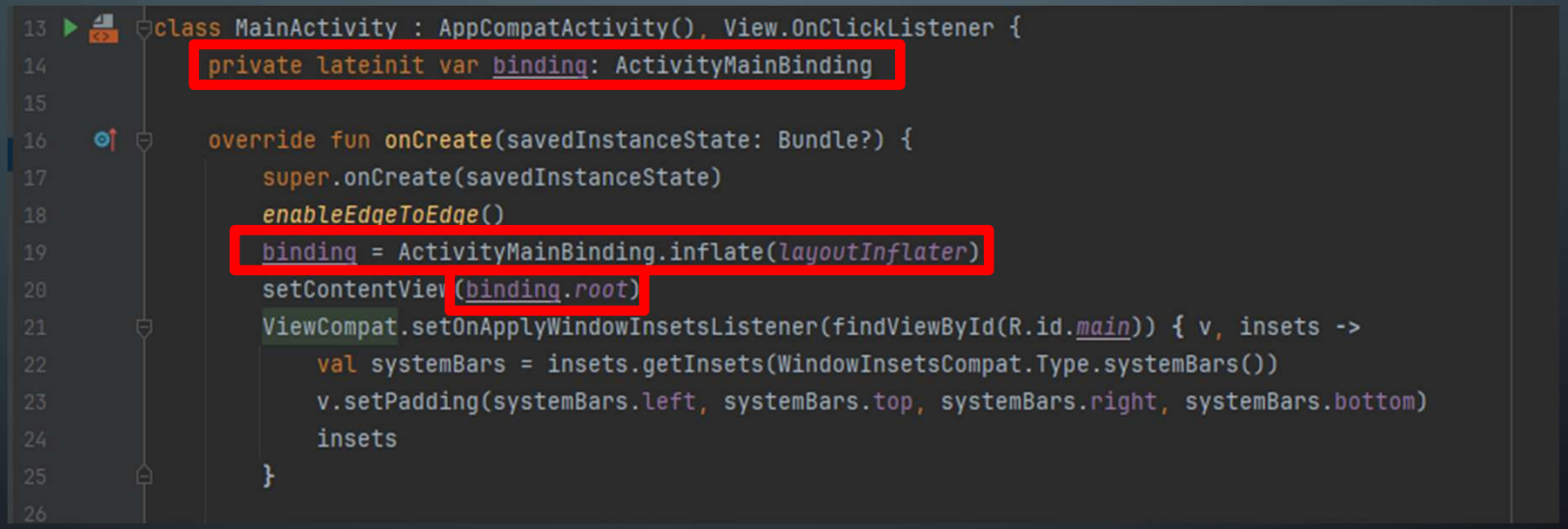
- Como nosso APP vai acessar a internet, precisamos dar esse direito nas configurações dele.
- Acesse o arquivo AndroidManifest.xml e acrescente a linha abaixo:





ACESSANDO API DE CEP NO ANDROID STUDIO

- Instancie um objeto para carregar os componentes visuais, para isso faça as alterações abaixo no arquivo MainActivity.kt



```
13  class MainActivity : AppCompatActivity(), View.OnClickListener {
14      private lateinit var binding: ActivityMainBinding
15
16      override fun onCreate(savedInstanceState: Bundle?) {
17          super.onCreate(savedInstanceState)
18          enableEdgeToEdge()
19          binding = ActivityMainBinding.inflate(layoutInflater)
20          setContentView(binding.root)
21          ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
22              val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
23              v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
24              insets
25          }
26
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Para tratar os eventos de click no botão, faça as alterações abaixo:

```
8   import com.tadseinstein.consultacep.databinding.ActivityMainBinding
9   import android.view.View
10
11  class MainActivity : AppCompatActivity(), View.OnClickListener {
12      private lateinit var binding: ActivityMainBinding
13
14      override fun onCreate(savedInstanceState: Bundle?) {
15          super.onCreate(savedInstanceState)
16          enableEdgeToEdge()
17          setContentView(binding.root)
18          ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
19              val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
20              v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
21              insets
22          }
23
24          binding.buttonProcuraCEP.setOnClickListener(this)
25      }
26
27      override fun onClick(view: View) {
28
29      }
30  }
```

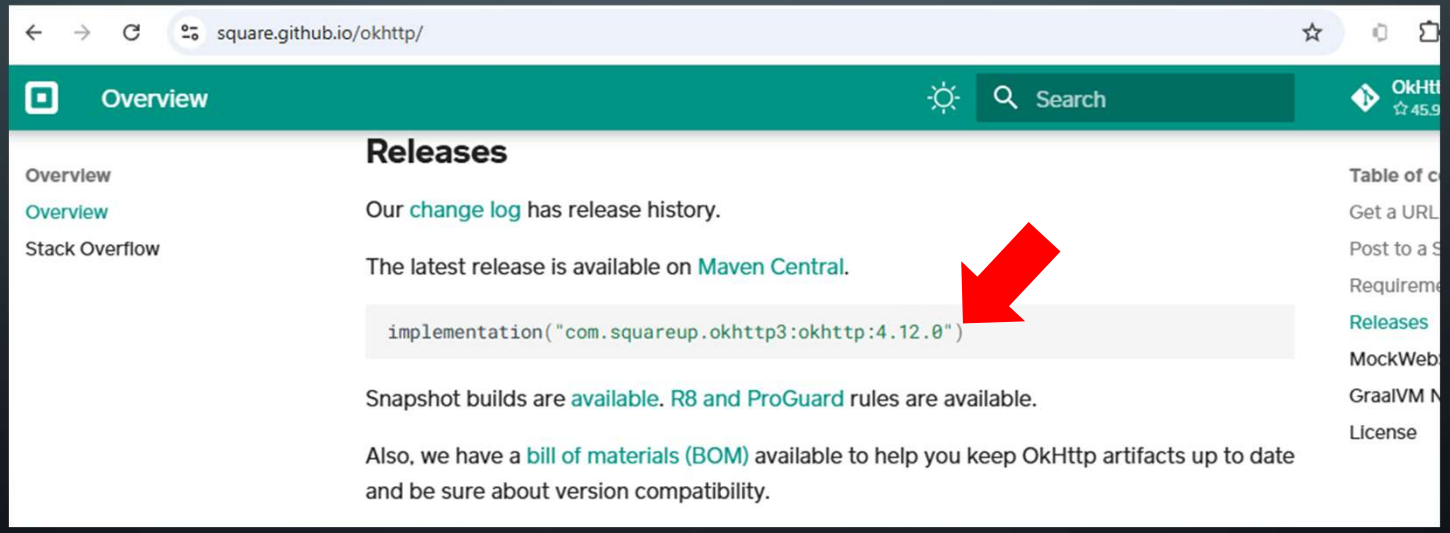
ACESSANDO API DE CEP NO ANDROID STUDIO

- Vamos criar uma função para tratar o click do botão de procura de CEP e chamar o mesmo da função onClick. Se tivermos mais botões é necessário fazer isso para cada um deles:

```
27    override fun onClick(view: View) {
28        if (view.id == R.id.button_procuraCEP) {
29            btProcCEP()
30        }
31    }
32
33    private fun btProcCEP() {
34
35    }
```

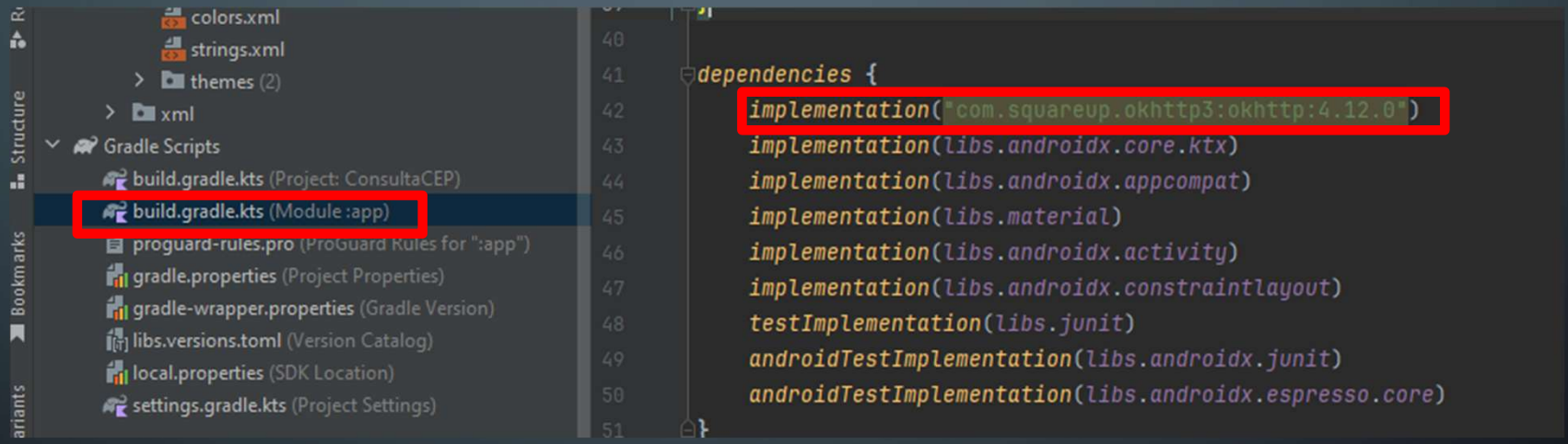
ACESSANDO API DE CEP NO ANDROID STUDIO

- Para facilitar o acesso a API vamos utilizar a biblioteca OkHttp no Android Studio.
- Site da biblioteca: https://square.github.io/okhttp/
- Acesse o site e role a pagina até a parte abaixo:

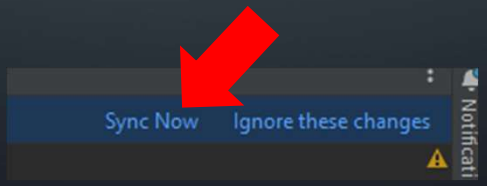


ACESSANDO API DE CEP NO ANDROID STUDIO

- Copie aquela linha indicada no slide anterior para o arquivo build.grade.kts(Module:app) na parte de dependências:



- Clique em Sync Now:



ACESSANDO API DE CEP NO ANDROID STUDIO

- Agora vamos criar o código para consulta do CEP. Dentro da função btProcCEP que criamos coloque o código abaixo:

```
37      private fun btProcCEP() {
38          val client = OkHttpClient()
39
40          val url = "https://viacep.com.br/ws/" + binding.editCep.text.toString() + "/json/"
41
42          var request = Request.Builder().url(url).get().build()
43
44          Thread {
45              val response = client.newCall(request).execute()
46              val jsonObj = JSONObject(response.body?.string())
47
48              runOnUiThread {
49                  binding.editRua.setText( jsonObj.getString( name: "logradouro") )
50                  binding.editBairro.setText( jsonObj.getString( name: "bairro") )
51                  binding.editCidade.setText( jsonObj.getString( name: "localidade") )
52                  binding.editUf.setText( jsonObj.getString( name: "uf") )
53                  binding.editNumero.requestFocus()
54              }
55          }.start()
56      }
```

ACESSANDO API DE CEP NO ANDROID STUDIO

- Entendendo o código do slide anterior:

```
val client = OkHttpClient()
```

  - Cria um cliente da biblioteca OkHttp para acessar a API

```
val url = "https://viacep.com.br/ws/" + binding.editCep.text.toString() + "/json/"
```

  - Define a URL já colocando o CEP digitado dentro do endereço

```
var request = Request.Builder().url(url).get().build()
```

  - Constrói a requisição que será enviada para o servidor da API

```
Thread {
```

  - Criamos uma thread secundária pois o Android não permite acesso a internet a partir da thread principal. Esse processo serve para impedir que um aplicativo pareça travado enquanto espera a resposta do servidor, que em alguns casos pode demorar um pouco.

ACESSANDO API DE CEP NO ANDROID STUDIO

```
val response = client.newCall(request).execute()
```

- Utiliza o cliente gerado para fazer a requisição a API e receber a resposta

```
val jsonObj = JSONObject(response.body?.string())
```

- Pega o conteúdo recebido (corpo da pagina em Json) e carrega em uma variável Json utilizando a classe JSONObject.

```
runOnUiThread {
```

- Faz com que os próximos comandos sejam executados na thread principal. O Android não permite alteração em campos da tela a partir de threads secundárias.

ACESSANDO API DE CEP NO ANDROID STUDIO

```
binding.editRua.setText( jsonObj.getString( name: "logradouro") )
binding.editBairro.setText( jsonObj.getString( name: "bairro") )
binding.editCidade.setText( jsonObj.getString( name: "localidade") )
binding.editUf.setText( jsonObj.getString( name: "uf") )
binding.editNumero.requestFocus()
```

- Preenche os editText com os valores carregados da API e coloca o foco no campo editNumero

ACESSANDO API DE CEP NO ANDROID STUDIO

- Pronto, só rodar o programa, colocar o CEP desejado e clicar em buscar:

